Raport — Zadanie 1: Wyszukiwanie najkrótszych ścieżek w sieci Kolei Dolnośląskich

Przedmiot: Sztuczna inteligencja i inżynieria wiedzy **Dane:** GTFS Koleje Dolnośląskie, ważne 03.03.2026–12.12.2026

1. Tło teoretyczne

1.1 Problem wyszukiwania ścieżki w transporcie publicznym

Sieć połączeń kolejowych modelowana jest jako **skierowany graf zależny od czasu**. Każdy wierzchołek reprezentuje przystanek (stację), a każda krawędź — jeden odcinek kursu pociągu między dwoma kolejnymi przystankami. Krawędź posiada dwa atrybuty czasowe: czas odjazdu i czas przyjazdu. Pasażer może skorzystać z krawędzi tylko wtedy, gdy przybywa na przystanek **nie później** niż czas odjazdu danego kursu.

Dla każdej pary kolejnych przystanków w kursie tworzona jest krawędź zawierająca: przystanek źródłowy, przystanek docelowy, czas odjazdu i czas przyjazdu.

Przesiadka modelowana jest implicitnie — pasażer czeka na przystanku do najbliższego dostępnego odjazdu w kierunku celu.

Graf jest zależny od czasu — w przeciwieństwie do klasycznego grafu ważonego, koszt przejścia krawędzią zależy nie tylko od jej wagi, ale też od momentu przybycia do wierzchołka. Pasażer, który przyjedzie za późno, musi poczekać na następny kurs.

Struktura danych — przykład:

```
# Węzły (stop_id → znormalizowane do parent_station jeśli istnieje):
"stop_lubawka", "stop_sedzislaw", "stop_walbrzych", "stop_rokitki"

# Graf sąsiedztwa: stop_id → lista krawędzi wychodzących
graph = {
    "stop_lubawka": [
        Connection(trip_id="D66_1", from_stop_id="stop_lubawka",
                   to_stop_id="stop_sedzislaw", departure_time=34260,
arrival_time=35460),
        Connection(trip_id="D66_2", from_stop_id="stop_lubawka",
                   to_stop_id="stop_sedzislaw", departure_time=41400,
arrival_time=42600),
    ],
    "stop_sedzislaw": [
        Connection(trip_id="D60_1", from_stop_id="stop_sedzislaw",
                   to_stop_id="stop_walbrzych", departure_time=35760,
arrival_time=37020),
        ...
    ],
    ...
}
```

```
# Jeden węzeł per stacja — wszystkie kursy w jednej liście.
# Algorytm filtruje: conn.departure_time >= current_arrival_time
```

Czasy podawane są w sekundach od północy: `09:31:00` = 9 * 3600 + 31 * 60 = 34260 s.

1.2 Algorytm Dijkstry

Algorytm Dijkstry znajduje najkrótszą ścieżkę w grafie ważonym z nieujemnymi wagami. Operuje na kolejce priorytetowej, z której zawsze wybierany jest wierzchołek o najniższym dotychczasowym koszcie `g(v)`.

Inicjalizacja: koszt wierzchołka startowego `g(s) = t_start`, pozostałych `g(v) = inf`.

Relaksacja krawędzi — dla każdego sąsiada `u` bieżącego wierzchołka `v`: jeśli `departure(v→u) >= g(v)` oraz `arrival(v→u) < g(u)`, to `g(u) = arrival(v→u)`.

Warunek `departure(v→u) >= g(v)` zapewnia, że pasażer zdąży na dany kurs. Algorytm kończy się gdy cel zostaje wyciągnięty z kolejki — w tym momencie `g(cel)` jest minimalnym możliwym czasem przybycia.

1.3 Algorytm A*

A* jest rozszerzeniem Dijkstry o funkcję heurystyczną `h(v)`, która szacuje minimalny koszt dotarcia z wierzchołka `v` do celu. Priorytet w kolejce wyznaczany jest przez:

```
f(v) = g(v) + h(v)
```

gdzie `g(v)` to rzeczywisty koszt dotychczasowej ścieżki, a `h(v)` to szacunek pozostałego kosztu.

Warunek dopuszczalności heurystyki: `h(v) <= h*(v)`, gdzie `h*(v)` to rzeczywisty minimalny koszt dotarcia z `v` do celu. Gwarantuje to, że A* znajdzie rozwiązanie optymalne.

Węzły w złym kierunku od celu otrzymują duże `h`, przez co trafiają na koniec kolejki i naturalnie nie są eksplorowane jeśli cel zostanie znaleziony wcześniej — stąd przyspieszenie względem Dijkstry. Gdy `h = 0`, A* degeneruje się do Dijkstry.

1.4 A* z kryterium czasu — heurystyka euklidesowa (`at`)

Koszt `g(v)` to czas przybycia w sekundach. Heurystyka szacuje minimalny czas dotarcia z przystanku `v` do celu jako odległość euklidesową w linii prostej podzieloną przez maksymalną prędkość pociągu (44.4 m/s ≈ 160 km/h):

```
h(v) = odleglosc_euklid(v, cel) / 44.4
```

Odległość euklidesowa liczona jest ze współrzędnych geograficznych WGS84 z przeliczeniem stopni na metry:

```
d = sqrt((delta_lat * 111320)^2 + (delta_lon * 111320 * cos(avg_lat))^2)
```

Heurystyka jest dopuszczalna, ponieważ pociąg nigdy nie pokona odległości między dwoma punktami szybciej niż jadąc 160 km/h w linii prostej.

1.5 A* z kryterium czasu — heurystyka oparta na odwróconej Dijkstrze (`ats`)

Heurystyka euklidesowa jest dopuszczalna, ale zgrubna — pociąg nigdy nie jedzie w linii prostej. Lepsza heurystyka powinna uwzględniać rzeczywistą topologię sieci.

Odwrócona Dijkstra od celu — przed startem algorytmu uruchamiamy Dijkstrę wstecz od stacji docelowej na uproszczonym grafie, gdzie waga krawędzi to minimalny czas przejazdu między przystankami (ignorujemy rozkład jazdy i czasy oczekiwania):

Waga krawędzi to minimum czasu przejazdu ze wszystkich kursów obsługujących daną parę przystanków: `w_min(u→v) = min(arrival_v - departure_u)`.

Wynik `dist[v]` stanowi dolne ograniczenie rzeczywistego czasu dotarcia z `v` do celu. Heurystyka jest dopuszczalna, ponieważ:

- ignoruje czasy oczekiwania na przesiadki (zawsze >= 0)
- używa minimalnych czasów przejazdu (rzeczywiste czasy są >= minimum)

Prekomputacja wykonywana jest raz przed startem wyszukiwania.

1.6 A* z kryterium przesiadek (`ap`)

Koszt `g(v)` jest krotką `(p, t)`, gdzie `p` to liczba przesiadek, `t` to czas przybycia. Sortowanie leksykograficzne — najpierw minimalizujemy przesiadki, przy remisie minimalizujemy czas.

Stan algorytmu rozszerzony jest o aktualny kurs: `v = (stop_id, trip_id)`. Przesiadka naliczana jest gdy pasażer zmienia `trip_id`.

Heurystyka opiera się na pytaniu: czy obecny kurs dojedzie do celu bez żadnej przesiadki?

- `h(v) = 0` jeśli `trip_id` należy do kursów docierających do celu
- `h(v) = 1` w przeciwnym razie

Jest to heurystyka dopuszczalna — jeśli obecny kurs nie dociera do celu, co najmniej jedna przesiadka jest nieunikniona.

Priorytet w kolejce: `f = (g_p + h, g_t)` — heurystyka dodawana jest wyłącznie do składowej przesiadkowej krotki, nie do czasu.

1.7 A* z kryterium przesiadek — heurystyka oparta na BFS po kursach (`aps`)

Heurystyka `ap` zwraca tylko 0 lub 1 — nie rozróżnia sytuacji gdy do celu potrzeba 2 lub więcej przesiadek. Lepsza heurystyka powinna szacować minimalną liczbę przesiadek dla każdego kursu.

Graf przesiadek — dwa kursy są połączone jeśli dzielą wspólny przystanek (można się między nimi przesiąść). BFS od kursów docierających bezpośrednio do celu daje minimalną liczbę przesiadek dla każdego kursu:

`min_transfers[trip]` = minimalna liczba przesiadek z kursu `trip` do celu.

Prekomputacja w dwóch krokach:

1. Dla każdego przystanku zbieramy wszystkie kursy które się tam zatrzymują (`stop_to_trips`)

2. BFS od kursów z `min_transfers = 0` — dla każdego sąsiedniego kursu (dzielącego przystanek) ustawiamy `min_transfers = d + 1`

Heurystyka dla stanu `v = (stop_id, trip_id)`:

- `h(v) = 0` jeśli `stop_id` jest celem
- `h(v) = min_transfers[trip_id]` jeśli `trip_id` jest znane
- `h(v) = min(min_transfers[c.trip_id] for c in connections(stop_id))` jeśli `trip_id = None` (stan startowy)

Jest dopuszczalna — `minTransfers` to dolne ograniczenie, ponieważ ignoruje czasy rozkładu (zakłada że zawsze można się przesiąść).

---

2. Implementacja

2.1 Przygotowanie danych GTFS

Dane wejściowe to pliki GTFS Kolei Dolnośląskich. Przed uruchomieniem algorytmu należy ustalić, które kursy są aktywne w podanym dniu, a następnie zbudować graf połączeń.

Fragment 1 — filtrowanie aktywnych kursów (`gtfs_loader.py`)

```
def load_active_service_ids(travel_date: date) -> set[ServiceId]:
    day_column = ["monday", ..., "sunday"][travel_date.weekday()]  # (1)
    date_int = int(travel_date.strftime("%Y%m%d"))                 # (2)
    active: set[ServiceId] = set()

    with open(GTFS_DIR / "calendar.txt") as f:
        for row in csv.DictReader(f):
            if (int(row["start_date"]) <= date_int <= int(row["end_date"])
# (3)
                    and row[day_column] == "1"):
# (4)
                active.add(row["service_id"])

    with open(GTFS_DIR / "calendar_dates.txt") as f:
        for row in csv.DictReader(f):
            if int(row["date"]) == date_int:
                if row["exception_type"] == "1":
                    active.add(row["service_id"])      # (5)
                elif row["exception_type"] == "2":
                    active.discard(row["service_id"])  # (6)

    return active
```

- **(1)** Wyznacza kolumnę dnia tygodnia w `calendar.txt` (np. `"monday"`) na podstawie obiektu `date`.
- **(2)** Konwertuje datę do formatu `YYYYMMDD` (integer) — format używany w GTFS.
- **(3)** Sprawdza czy data podróży mieści się w przedziale ważności wzorca kursowania.
- **(4)** Sprawdza czy wzorzec jest aktywny w danym dniu tygodnia.

- **(5)** `exception_type=1` — kurs dodany wyjątkowo w tej dacie (np. dodatkowy kurs świąteczny).
- **(6)** `exception_type=2` — kurs odwołany w tej dacie (np. zawieszenie z powodu remontu).

Wyjątki z `calendar_dates.txt` **nadpisują** wzorzec tygodniowy — wywołanie `discard` usuwa service_id który był wcześniej dodany przez wzorzec.

Fragment 2 — budowanie grafu połączeń (`gtfs_loader.py`)

```
def load_connections(active_trip_ids: set[TripId], time_offset: Seconds =
0) -> list[Connection]:
    norm = load_stop_normalization()              # (1)
    trip_stops: dict[TripId, list[StopTimeRow]] = defaultdict(list)

    with open(GTFS_DIR / "stop_times.txt") as f:
        for row in csv.DictReader(f):
            if row["trip_id"] not in active_trip_ids:  # (2)
                continue
            trip_stops[row["trip_id"]].append(StopTimeRow(
                sequence=int(row["stop_sequence"]),
                stop_id=norm.get(row["stop_id"], row["stop_id"]),  # (3)
                arrival_time=time_to_seconds(row["arrival_time"]) +
time_offset,   # (4)
                departure_time=time_to_seconds(row["departure_time"]) +
time_offset,
                pickup_type=int(row.get("pickup_type") or 0),
            ))

    connections = []
    for trip_id, stop_times in trip_stops.items():
        sorted_stop_times = sorted(stop_times, key=lambda v: v.sequence)
# (5)
        for i in range(len(sorted_stop_times) - 1):
            from_v, to_v = sorted_stop_times[i], sorted_stop_times[i + 1]
            if from_v.pickup_type == 1:
# (6)
                continue
            connections.append(Connection(
                trip_id=trip_id,
                from_stop_id=from_v.stop_id,
                to_stop_id=to_v.stop_id,
                departure_time=from_v.departure_time,
                arrival_time=to_v.arrival_time,
            ))
    return connections
```

- **(1)** Normalizacja peronów do stacji nadrzędnej — peron `2246926` (np. "Wrocław Gł. peron 1") mapowany jest do `parent_station`. Dzięki temu przesiadki między peronami tej samej stacji działają poprawnie.
- **(2)** Wczytujemy tylko kursy aktywne w podanym dniu — pomijamy resztę pliku.

- **(3)** `norm.get(stop_id, stop_id)` — jeśli przystanek ma stację nadrzędną, używamy jej ID; w przeciwnym razie zostawiamy oryginalne ID.
- **(4)** `time_offset=86400` używany jest przy wczytywaniu kursów następnego dnia — czas przesuwa się o 24h, dzięki czemu podróże przekraczające północ obsługiwane są poprawnie (np. kurs o 23:50 z przyjazdem o 00:30 następnego dnia).
- **(5)** Przystanki kursu sortowane po `stop_sequence` — gwarantuje poprawną kolejność nawet jeśli wiersze w pliku są nieuporządkowane.
- **(6)** `pickup_type=1` oznacza brak możliwości wsiadania — pomijamy takie krawędzie. Wysiadanie jest zawsze możliwe, więc `drop_off_type` nie jest sprawdzany.

2.2 Wzorzec SearchConfig — wspólny algorytm dla wszystkich kryteriów

Zamiast implementować osobno Dijkstrę i A*, zastosowano wzorzec konfiguracji. `SearchConfig` (zdefiniowany w `models.py`) definiuje kryterium optymalizacji jako zestaw funkcji, a właściwy algorytm (`search` w `dijkstra.py`) jest wspólny dla wszystkich wariantów. Wszystkie fabryki konfiguracji (`make_time_config`, `make_transfers_config` i warianty A*) zgrupowane są w `configs.py`.

```
@dataclass
class SearchConfig:
    initial_states: Callable[[set[StopId], Seconds], list[tuple[Cost,
State]]]  # (1)
    expand:         Callable[[State, Cost, Graph], list[tuple[Cost, State,
Connection]]]  # (2)
    is_goal:        Callable[[State, set[StopId]], bool]   # (3)
    get_stop_id:    Callable[[State], StopId]              # (4)
    get_arrival_time: Callable[[Cost], Seconds]            # (5)
    heuristic: Callable[[State], Any] | None = None  # (6)
    make_f:    Callable[[Cost, Any], Any] | None = None           # (7)
    on_visit:  Callable[[int, State, Cost], None] | None = None   # (8)
```

- **(1)** `initial_states` — generuje stany startowe z kosztem początkowym. Dla kryterium czasu: `(czas_odjazdu, stop_id)`. Dla przesiadek: `((0, czas_odjazdu), (stop_id, None))`.
- **(2)** `expand` — generuje sąsiadów stanu. Zawiera logikę filtrowania krawędzi (np. warunek zdążenia na pociąg).
- **(3)** `is_goal` — sprawdza czy osiągnięto cel. Dla czasu: `state in target_ids`. Dla przesiadek: `state[0] in target_ids` (bo stan to krotka).
- **(4)** `get_stop_id` — wyłuskuje `stop_id` ze stanu (różne dla różnych typów stanu).
- **(5)** `get_arrival_time` — wyłuskuje czas przybycia z kosztu (różne dla `int` i krotki).
- **(6)** `heuristic` — funkcja heurystyczna `h(v)`. Gdy `None`, algorytm działa jak Dijkstra.
- **(7)** `make_f` — definiuje jak łączyć `g` z `h` przy obliczaniu priorytetu. Potrzebne gdy koszt jest krotką — nie można po prostu dodać `(p, t) + 1`.
- **(8)** `on_visit` — opcjonalny callback diagnostyczny wywoływany przy każdej wizycie węzła.

Fragment 3 — Dijkstra z kryterium czasu (`t`)

```
def make_time_config() -> SearchConfig:
    return SearchConfig(
        initial_states=lambda source_ids, t: [(t, sid) for sid in
source_ids],  # (1)
        expand=lambda state, cost, graph: [
            (conn.arrival_time, conn.to_stop_id, conn)
            for conn in graph.get(state, [])
            if conn.departure_time >= cost    # (2)
        ],
        is_goal=lambda state, target_ids: state in target_ids,
        get_stop_id=lambda state: state,
        get_arrival_time=lambda cost: cost,
    )
```

- **(1)** Każda stacja źródłowa startuje z kosztem równym najwcześniejszemu czasowi odjazdu. Możliwe jest wiele stacji źródłowych (różne perony tej samej stacji).
- **(2)** Kurs jest dostępny tylko jeśli odjeżdża nie wcześniej niż czas przybycia pasażera. Warunek ten modeluje oczekiwanie na przystanku.

Fragment 4 — Dijkstra z kryterium przesiadek (`p`)

```
def make_transfers_config() -> SearchConfig:
    def expand(state: tuple, cost: tuple, graph: Graph) -> list:
        stop_id, current_trip = state          # (1)
        num_transfers, current_time = cost     # (2)
        result = []
        for conn in graph.get(stop_id, []):
            if conn.departure_time < current_time:
                continue
            is_transfer = current_trip is not None and conn.trip_id !=
current_trip  # (3)
            new_cost = (num_transfers + (1 if is_transfer else 0),
conn.arrival_time)  # (4)
            result.append((new_cost, (conn.to_stop_id, conn.trip_id),
conn))
        return result

    return SearchConfig(
        initial_states=lambda source_ids, t: [((0, t), (sid, None)) for
sid in source_ids],  # (5)
        expand=expand,
        is_goal=lambda state, target_ids: state[0] in target_ids,
        get_stop_id=lambda state: state[0],
        get_arrival_time=lambda cost: cost[1],
    )
```

- **(1)** Stan rozszerzony o `trip_id` — wiemy którym pociągiem aktualnie jedziemy.

- **(2)** Koszt to krotka `(liczba_przesiadek, czas_przybycia)` — sortowanie leksykograficzne minimalizuje najpierw przesiadki.
- **(3)** Przesiadka naliczana jest gdy zmieniamy `trip_id`. Warunek `current_trip is not None` obsługuje stan startowy (jeszcze nie wsiedliśmy do żadnego pociągu).
- **(4)** Nowy koszt: jeśli przesiadka — `num_transfers + 1`, w przeciwnym razie bez zmiany.
- **(5)** Stan startowy: `trip_id = None` — nie jesteśmy jeszcze w żadnym pociągu.

---

*Fragment 5 — A z kryterium czasu, heurystyka euklidesowa (`at`)**

```
def make_astar_time_euclidean_config(coords, target_ids,
max_speed_ms=44.4) -> SearchConfig:
    target_coords = [coords[sid] for sid in target_ids if sid in coords]
# (1)

    def heuristic(state: StopId) -> Seconds:
        if state not in coords or not target_coords:
            return 0                                    # (2)
        lat1, lon1 = coords[state]
        min_dist = min(
            math.sqrt(
                ((lat2 - lat1) * 111_320) ** 2
                + ((lon2 - lon1) * 111_320 * math.cos(math.radians((lat1 +
lat2) / 2))) ** 2
            )
            for lat2, lon2 in target_coords             # (3)
        )
        return int(min_dist / max_speed_ms)             # (4)

    config = make_time_config()
    config.heuristic = heuristic
    return config
```

- **(1)** Prekomputacja współrzędnych stacji docelowych — unikamy wielokrotnego wyszukiwania podczas wyszukiwania.
- **(2)** Gdy brak danych geograficznych, heurystyka zwraca 0 — A* degeneruje się do Dijkstry dla tego węzła, co jest bezpieczne.
- **(3)** Dla zestawu stacji docelowych (różne perony) bierzemy minimalną odległość.
- **(4)** Dzielimy przez prędkość wyrażoną w m/s — wynik w sekundach. `int()` zaokrągla w dół, co gwarantuje niedoszacowanie.

---

*Fragment 6 — A z kryterium czasu, heurystyka oparta na odwróconej Dijkstrze (`ats`)**

```
def make_astar_time_reverse_dijkstra_config(graph: Graph, target_ids:
set[StopId]) -> SearchConfig:
    # Krok 1: minimalne czasy przejazdu między przystankami
    min_travel: dict[tuple[StopId, StopId], int] = {}
    for conns in graph.values():
```

```
        for conn in conns:
            key = (conn.from_stop_id, conn.to_stop_id)
            t = conn.arrival_time - conn.departure_time       # (1)
            if key not in min_travel or t < min_travel[key]:
                min_travel[key] = t

    # Krok 2: odwrócony graf
    rev: dict[StopId, list[tuple[StopId, int]]] = defaultdict(list)
    for (from_stop, to_stop), t in min_travel.items():
        rev[to_stop].append((from_stop, t))                   # (2)

    # Krok 3: Dijkstra wstecz od celu
    dist: dict[StopId, int] = {}
    queue = []
    for tid in target_ids:
        dist[tid] = 0
        heapq.heappush(queue, (0, tid))                       # (3)

    while queue:
        d, v = heapq.heappop(queue)
        if d > dist.get(v, float("inf")):
            continue
        for u, t in rev.get(v, []):
            nd = d + t
            if nd < dist.get(u, float("inf")):
                dist[u] = nd
                heapq.heappush(queue, (nd, u))                # (4)

    def heuristic(state: StopId) -> Seconds:
        return dist.get(state, 0)                             # (5)

    config = make_time_config()
    config.heuristic = heuristic
    return config
```

- **(1)** Czas przejazdu na odcinku = `arrival - departure`. Bierzemy minimum ze wszystkich kursów obsługujących daną parę przystanków — dolne ograniczenie.
- **(2)** Odwrócenie krawędzi: zamiast `A → B`, tworzymy `B → A`. Pozwala na Dijkstrę "wstecz" od celu.
- **(3)** Inicjalizacja kolejki od wszystkich stacji docelowych z kosztem 0 — wieloźródłowa Dijkstra.
- **(4)** Standardowa relaksacja Dijkstry na odwróconym grafie. Wynik `dist[v]` to minimalna suma czasów przejazdu z `v` do celu, ignorując oczekiwanie.
- **(5)** Heurystyka to prekomputowana dolna granica. Dla przystanków poza siecią KD zwraca 0 (bezpieczne).

---

*Fragment 7 — A z kryterium przesiadek (`ap`)**

```
def make_astar_transfers_direct_trip_config(graph: Graph, target_ids:
set[StopId]) -> SearchConfig:
    # Prekomputacja: które kursy docierają do stacji docelowej?
    trips_to_target: set[str] = set()
```

```
    for conns in graph.values():
        for conn in conns:
            if conn.to_stop_id in target_ids:
                trips_to_target.add(conn.trip_id)            # (1)

    def heuristic(state: tuple) -> int:
        stop_id, trip_id = state
        if stop_id in target_ids:
            return 0                                          # (2)
        if trip_id is not None and trip_id in trips_to_target:
            return 0                                          # (3)
        return 1                                              # (4)

    config = make_transfers_config()
    config.heuristic = heuristic
    config.make_f = lambda cost, h: (cost[0] + h, cost[1])   # (5)
    return config
```

- **(1)** Prekomputacja liniowa — dla każdej krawędzi kończącej się w celu zapisujemy `trip_id`.
- **(2)** Już na miejscu — 0 przesiadek więcej.
- **(3)** Obecny kurs ma zatrzymanie w stacji docelowej — dotrzemy bez przesiadki.
- **(4)** Obecny kurs nie dociera do celu — konieczna co najmniej 1 przesiadka.
- **(5)** `make_f` definiuje jak dodać heurystykę do krotki kosztu — dodajemy tylko do składowej przesiadkowej, nie do czasu.

---

*Fragment 7b — A z kryterium przesiadek, heurystyka BFS po kursach (`aps`)**

```
def make_astar_transfers_bfs_config(graph: Graph, target_ids: set[StopId])
-> SearchConfig:
    # Krok 1: dwa indeksy — przystanek→kursy i kurs→przystanki
    stop_to_trips: dict[StopId, set[TripId]] = defaultdict(set)
    trip_to_stops: dict[TripId, set[StopId]] = defaultdict(set)
    for conns in graph.values():
        for conn in conns:
            stop_to_trips[conn.from_stop_id].add(conn.trip_id)        #
(1)
            trip_to_stops[conn.trip_id].add(conn.from_stop_id)        #
(1)

    # Krok 2: BFS od kursów docierających bezpośrednio do celu
    min_transfers: dict[TripId, int] = {}
    queue: deque = deque()
    for conns in graph.values():
        for conn in conns:
            if conn.to_stop_id in target_ids and conn.trip_id not in
min_transfers:
                min_transfers[conn.trip_id] = 0                       #
(2)
                queue.append(conn.trip_id)
```

```
    while queue:
        trip = queue.popleft()
        d = min_transfers[trip]
        for stop_id in trip_to_stops[trip]:                          #
(3)
            for other_trip in stop_to_trips[stop_id]:
                if other_trip not in min_transfers:
                    min_transfers[other_trip] = d + 1                 #
(4)
                    queue.append(other_trip)

    def heuristic(state: tuple) -> int:
        stop_id, trip_id = state
        if stop_id in target_ids:
            return 0
        if trip_id is None:                                           #
(5)
            for conn in graph.get(stop_id, []):
                if min_transfers.get(conn.trip_id, 1) == 0:
                    return 0
            return 1
        return min_transfers.get(trip_id, 1)

    config = make_transfers_config()
    config.heuristic = heuristic
    config.make_f = lambda cost, h: (cost[0] + h, cost[1])
    return config
```

- **(1)** Dwa indeksy: `stop_to_trips` (przystanek → kursy zatrzymujące się tam) i `trip_to_stops` (kurs → przystanki na których staje). Pozwalają na O(1) lookup zamiast przeszukiwania całego grafu.
- **(2)** Inicjalizacja BFS — kursy docierające bezpośrednio do celu mają 0 przesiadek.
- **(3)** Dla każdego przystanku kursu szukamy kursów z którymi można się tam przesiąść — dzięki `trip_to_stops` nie trzeba przeszukiwać całego grafu.
- **(4)** Relaksacja BFS — kurs sąsiedni (dzielący przystanek) potrzebuje o 1 przesiadkę więcej. Warunek `not in min_transfers` gwarantuje że BFS zapisuje zawsze najkrótszą drogę (pierwszy dotarcie = najmniejsza liczba przesiadek).
- **(5)** Stan startowy ma `trip_id = None` — sprawdzamy czy z przystanku odjeżdża bezpośredni kurs do celu. Bez tej korekty heurystyka zwracałaby 1 nawet gdy istnieje połączenie bezpośrednie, co byłoby przeszacowaniem.

Fragment 8 — wspólny algorytm wyszukiwania

```
def search(graph, source_ids, target_ids, departure_time, config):
    h = config.heuristic

    best_cost: dict[State, Cost] = {}
    prev: dict[State, tuple[State, Connection] | None] = {}
    queue = []
    counter = itertools.count()    # (1)
```

```
    for cost, state in config.initial_states(source_ids, departure_time):
        best_cost[state] = cost
        prev[state] = None
        if h:
            h_val = h(state)
            f = config.make_f(cost, h_val) if config.make_f else cost +
h_val
        else:
            f = cost                                            # (2)
        heapq.heappush(queue, (f, next(counter), cost, state))

    while queue:
        _, _, current_cost, current_state = heapq.heappop(queue)

        if current_cost != best_cost.get(current_state):      # (3)
            continue

        if config.is_goal(current_state, target_ids):
            return _build_result(...), step                    # (4)

        for new_cost, new_state, conn in config.expand(current_state,
current_cost, graph):
            if new_state not in best_cost or new_cost <
best_cost[new_state]:
                best_cost[new_state] = new_cost
                prev[new_state] = (current_state, conn)        # (5)
                if h:
                    h_val = h(new_state)
                    new_f = config.make_f(new_cost, h_val) if
config.make_f else new_cost + h_val
                else:
                    new_f = new_cost
                heapq.heappush(queue, (new_f, next(counter), new_cost,
new_state))

    return None, step
```

- **(1)** `itertools.count()` jako tiebreaker — gdy dwa węzły mają równe `f`, heap porównuje kolejny element krotki. Counter gwarantuje unikalność i unika błędu porównywania stanów.
- **(2)** Gdy brak heurystyki (`None`), priorytet = koszt = klasyczna Dijkstra.
- **(3)** Lazy deletion — zamiast aktualizować istniejące wpisy w heap (kosztowne), dodajemy nowy wpis i ignorujemy zdezaktualizowane. Wpis jest zdezaktualizowany gdy `current_cost != best_cost[state]`.
- **(4)** Algorytm kończy się gdy cel **wychodzi z kolejki** — nie gdy jest do niej dodany. Gwarantuje optymalność: gdy cel jest na szczycie kolejki, żadna inna ścieżka nie może być lepsza.
- **(5)** `prev[new_state] = (current_state, conn)` przechowuje poprzedni stan i krawędź — umożliwia rekonstrukcję ścieżki przez cofanie się po `prev`.

---

3. Wyniki

Testy wykonano na danych Kolei Dolnośląskich, piątek, trasa Lubawka → Rokitki (odjazd 08:40). Kryterium: `t` = Dijkstra czas, `p` = Dijkstra przesiadki, `at` = A* czas (euklid), `ats` = A* czas (rev-Dijkstra), `ap` = A* przesiadki.

3.1 Lubawka → Rokitki, kryterium czasu (`t`)

```
$ python main.py "Lubawka" "Rokitki" t "8:40" "pt"

Lubawka → Sędzisław              [D66]  09:31:00 → 09:51:00
Sędzisław → Wałbrzych Fabryczny  [D60]  09:56:00 → 10:22:00
Wałbrzych Fabryczny → Jaworzyna Śląska  [D60]  10:24:00 → 10:53:00
Jaworzyna Śląska → Legnica       [D91]  10:59:00 → 11:49:00
Legnica → Chojnów                [D1]   12:14:00 → 12:26:00
Chojnów → Rokitki                [D14]  13:20:00 → 13:30:00

Trasa:       Lubawka → Rokitki
Odjazd:      09:31:00
Przyjazd:    13:30:00
Czas:        00:03:59
Przesiadki:  5
Linie:       ['D66', 'D60', 'D91', 'D1', 'D14']
Odwiedzone węzły: 239
```

Trasa prowadzi przez Wałbrzych i Legnicę — brak połączeń bezpośrednich między małymi stacjami. Algorytm znalazł 5 przesiadek jako koszt najszybszego dotarcia.

3.2 Lubawka → Rokitki, kryterium przesiadek (`p`)

```
$ python main.py "Lubawka" "Rokitki" p "8:40" "pt"

Lubawka → Sędzisław        [D66]  09:31:00 → 09:51:00
Sędzisław → Wrocław Główny [D60]  09:56:00 → 11:38:00
Wrocław Główny → Rokitki   [D14]  13:35:00 → 14:44:00

Trasa:       Lubawka → Rokitki
Odjazd:      09:31:00
Przyjazd:    14:44:00
Czas:        00:05:13
Przesiadki:  2
Linie:       ['D66', 'D60', 'D14']
Odwiedzone węzły: 2174
```

Kryterium przesiadek wybiera trasę przez Wrocław — 2 przesiadki zamiast 5, kosztem 1h 14min dłuższej podróży. Algorytm odwiedza 2174 węzłów (vs 239 dla `t`) — kryterium przesiadkowe przeszukuje znacznie więcej przestrzeni.

3.3 Lubawka → Rokitki, A* czas euklid (`at`) i rev-Dijkstra (`ats`)

```
$ python main.py "Lubawka" "Rokitki" at "8:40" "pt"

Lubawka → Sędzisław              [D66]  09:31:00 → 09:51:00
Sędzisław → Wałbrzych Fabryczny  [D60]  09:56:00 → 10:22:00
Wałbrzych Fabryczny → Jaworzyna Śląska  [D60]  10:24:00 → 10:53:00
Jaworzyna Śląska → Legnica       [D91]  10:59:00 → 11:49:00
Legnica → Chojnów                [D1]   12:14:00 → 12:26:00
Chojnów → Rokitki                [D14]  13:20:00 → 13:30:00

Trasa:       Lubawka → Rokitki
Przyjazd:    13:30:00
Przesiadki:  5
Odwiedzone węzły: 193
```

```
$ python main.py "Lubawka" "Rokitki" ats "8:40" "pt"

(identyczna trasa)
Odwiedzone węzły: 118
```

Obie wersje A* zwracają tę samą trasę co Dijkstra — wynik optymalny. Różnica jest w liczbie odwiedzonych węzłów: `at` redukuje o 19%, `ats` o 51%.

3.4 Lubawka → Rokitki, A* przesiadki (`ap`) i BFS po kursach (`aps`)

```
$ python main.py "Lubawka" "Rokitki" ap "8:40" "pt"

Lubawka → Sędzisław        [D66]  09:31:00 → 09:51:00
Sędzisław → Wrocław Główny [D60]  09:56:00 → 11:38:00
Wrocław Główny → Rokitki   [D14]  13:35:00 → 14:44:00

Trasa:       Lubawka → Rokitki
Przyjazd:    14:44:00
Przesiadki:  2
Odwiedzone węzły: 243
```

```
$ python main.py "Lubawka" "Rokitki" aps "8:40" "pt"

(identyczna trasa)
Odwiedzone węzły: 523
```

A* `ap` redukuje węzły o **89%** względem Dijkstry przesiadkowego. `aps` daje wynik identyczny, ale odwiedza więcej węzłów (523 vs 243).

Analiza logów (`--verbose`) ujawnia przyczynę: BFS w `aps` przypisał kursom jadącym w stronę Czech (Kralovec, Trutnov) `min_transfers=2`, a kluczowemu przystankowi Sędzisław `min_transfers=3`. Ponieważ `f = (przesiadki_g + h, czas)`, kierunek czeski dostaje `f=(2,...)` a Sędzisław `f=(3,...)` — algorytm eksploruje więc całą czeską gałąź zanim dotrze do Sędzisławia, gdzie jest właściwa przesiadka na Wrocław.

BFS oblicza `min_transfers` na uproszczonym grafie bez rozkładu jazdy — zakłada że zawsze można się przesiąść. W rzeczywistości kursy czeskie nie mają żadnego połączenia z Rokitkami, ale heurystyka tego nie wie i błędnie faworyzuje ten kierunek. Heurystyka pozostaje dopuszczalna (nie przeszacowuje), ale jest niedokładna topologicznie dla kursów wychodzących poza sieć KD.

3.5 Wrocław Główny → Karpacz 15:20, A* przesiadki (`ap` vs `aps`)

```
$ python main.py "Wrocław Główny" "Karpacz" ap "15:20" "pon"
Przesiadki:  1
Odwiedzone węzły: 786

$ python main.py "Wrocław Główny" "Karpacz" aps "15:20" "pon"
Przesiadki:  1
Odwiedzone węzły: 50
```

Dla dłuższej trasy (Wrocław → Karpacz) `aps` odwiedza **16× mniej węzłów** niż `ap`. Różnica wynika z topologii sieci — wiele kursów ma `min_transfers = 0` (docierają do Karpacza bezpośrednio przez linię D62), co pozwala heurystyce wcześnie odrzucić kursy idące w złą stronę.

3.6 Zbiorcze porównanie odwiedzonych węzłów

Kryterium	Algorytm	Węzły	Redukcja	Przyjazd	Przesiadki
czas	Dijkstra `t`	239	—	13:30	5
czas	A* euklid `at`	193	−19%	13:30	5
czas	A* rev-Dijkstra `ats`	118	**−51%**	13:30	5
przesiadki	Dijkstra `p`	2174	—	14:44	2
przesiadki	A* `ap`	243	**−89%**	14:44	2
przesiadki	A* BFS-kursy `aps` (Lubawka→Rokitki)	523	−76%	14:44	2
przesiadki	A* BFS-kursy `aps` (Wrocław→Karpacz)	50	**−94%** vs `ap`	—	1

Skuteczność `aps` zależy od topologii sieci. Dla tras gdzie wiele kursów ma `min_transfers > 1` (sieć rzadka, daleka trasa) — przewaga duża. Dla tras gdzie większość kursów ma `min_transfers = 1` —

heurystyki `ap` i `aps` dają podobne wartości i różnica zanika.

Zadanie 2 — TSP na sieci kolejowej (Tabu Search)

6. Tło teoretyczne

6.1 Problem komiwojażera (TSP) na sieci kolejowej

Problem komiwojażera (TSP) polega na znalezieniu najkrótszej zamkniętej trasy odwiedzającej zbiór miast dokładnie raz i wracającej do punktu startowego. Szukamy permutacji przystanków minimalizującej sumę kosztów kolejnych odcinków trasy.

W klasycznym TSP macierz odległości `d(i,j)` jest stała. W naszym przypadku — sieci kolejowej — koszt przejazdu między przystankami **zależy od czasu przyjazdu do przystanku pośredniego**, ponieważ dostępność pociągów zmienia się w zależności od rozkładu. Jest to **time-dependent TSP**.

Macierz kosztów nie jest prekomputowana — zamiast tego, przy każdej ocenie trasy wywołujemy `search()` z Zadania 1 sekwencyjnie dla kolejnych odcinków. Czas odjazdu każdego odcinka wynika z czasu przyjazdu odcinka poprzedniego.

Ograniczenie implementacji: algorytm nie wykrywa sytuacji gdy optymalna trasa A→B przejeżdża przez punkt C, który jest osobnym punktem na liście do odwiedzenia. W takim przypadku C zostanie "odwiedzony przy okazji", ale algorytm i tak wywoła osobno `search(B→C)` i `search(C→...)`, co może prowadzić do zbędnego nawracania. Jest to znane ograniczenie TSP na sieciach transportowych gdzie trasy nie są euklidesowe.

6.2 Przeszukiwanie lokalne i jego ograniczenia

Przeszukiwanie lokalne iteracyjnie zastępuje bieżące rozwiązanie `s` lepszym sąsiadem `s'` z sąsiedztwa `N(s)`. Kończy się gdy żaden sąsiad nie jest lepszy — w **optimum lokalnym**, które niekoniecznie jest globalnym.

Dla TSP z `n` miastami sąsiedztwo **swap** zawiera `n*(n-1)/2` kandydatów — każdą parę miast, które można ze sobą zamienić w kolejności odwiedzania.

6.3 Tabu Search

Tabu Search rozszerza przeszukiwanie lokalne o **listę tabu** `T` przechowującą ostatnio wykonane ruchy. W każdej iteracji:

1. Generujemy całe sąsiedztwo `N(s)` (lub jego próbkę).
2. Wybieramy **najlepszego** kandydata spoza listy tabu — nawet jeśli gorszy od bieżącego.
3. Dodajemy wykonany ruch do `T`.
4. Aktualizujemy globalne optimum jeśli nowe rozwiązanie jest lepsze.

Dzięki liście tabu algorytm **nie cofa się** do niedawno odwiedzonych rozwiązań i może wyjść z optimum lokalnego.

6.4 Warianty algorytmu

Wariant (a) — bazowy: lista tabu nieograniczona (brak limitu). Każdy ruch jest pamiętany na zawsze.

Wariant (b) — zmienny rozmiar T: rozmiar listy `= 2 * |L|`. Lista traktowana jako kolejka FIFO — przy przepełnieniu usuwany jest najstarszy ruch. Dla małych list L tabu jest krótkie (nie blokuje zbyt wiele), dla dużych — dłuższe.

Wariant (c) — kryterium aspiracji: ruch z listy tabu jest dopuszczony wyjątkowo, jeśli prowadzi do nowego globalnego optimum (koszt kandydata < dotychczasowe najlepsze). Zapobiega odrzucaniu potencjalnie świetnych ruchów tylko dlatego, że są na liście tabu.

Wariant (d) — próbkowanie sąsiedztwa: zamiast sprawdzać wszystkich `n*(n-1)/2` sąsiadów, losujemy `k` z nich. Redukuje czas obliczenia jednej iteracji z kwadratowego do liniowego względem `k`.

---

7. Implementacja

7.1 Rozwiązanie początkowe — greedy (`tabu.py`)

```
def greedy_initial(start_ids, candidates, start_time, graph):
    remaining = list(candidates)
    ordered = []
    current_ids = start_ids
    current_time = start_time

    while remaining:
        best_arrival = float('inf')
        best_idx = None

        for idx, (_, ids) in enumerate(remaining):          # (1)
            result, _ = search(graph, current_ids, ids,
                               current_time, make_time_config())
            if result is not None and result.arrival_time < best_arrival:
                best_arrival = result.arrival_time
                best_idx = idx

        if best_idx is None:                                 # (2)
            ordered.extend(remaining)
            break

        next_stop = remaining.pop(best_idx)                 # (3)
        ordered.append(next_stop)
        current_ids = next_stop[1]
        current_time = best_arrival

    return ordered
```

- **(1)** Dla każdego nieodwiedzonego przystanku uruchamiamy `search()` z bieżącej pozycji i czasu — zawsze kryterium czasu, bo greedy ma tylko znaleźć dobry punkt startowy.

- **(2)** Brak połączenia do żadnego przystanku — dołączamy pozostałe w oryginalnej kolejności (bezpieczny fallback).
- **(3)** Usuwamy wybrany przystanek z listy i aktualizujemy bieżącą pozycję i czas.

7.2 Ocena trasy (`tabu.py`)

```
def evaluate_tour(tour_ids, start_time, graph, criterion):
    current_time = start_time
    total_transfers = 0
    results = []

    for i in range(len(tour_ids) - 1):                          # (1)
        config = make_transfers_config() if criterion == 'p' \
                 else make_time_config()
        result, _ = search(graph, tour_ids[i], tour_ids[i + 1],
                           current_time, config)

        if result is None:
            return float('inf'), None                           # (2)

        results.append(result)
        current_time = result.arrival_time                      # (3)

        if criterion == 'p':
            legs = result.legs
            transfers = sum(
                1 for j in range(1, len(legs))
                if legs[j].trip_id != legs[j - 1].trip_id
            )
            total_transfers += transfers                        # (4)

    if criterion == 'p':
        cost = (total_transfers, current_time - start_time)    # (5)
    else:
        cost = current_time - start_time

    return cost, results
```

- **(1)** Iterujemy po kolejnych parach przystanków w trasie — każda para to osobne wywołanie `search()`.
- **(2)** Brak połączenia na którymkolwiek odcinku → koszt nieskończony, trasa niedopuszczalna.
- **(3)** Czas przyjazdu staje się czasem odjazdu następnego odcinka — kluczowe dla time-dependent TSP.
- **(4)** Zliczamy przesiadki w obrębie każdego odcinka. Przejście między odcinkami (przybycie do B, odjazd do C) jest obsługiwane naturalnie — `make_transfers_config` startuje z `trip_id=None`, więc pierwsze wsiadanie nie liczy się jako przesiadka.
- **(5)** Koszt dla kryterium `p` to krotka (przesiadki, czas) — leksykograficznie: najpierw minimalizuj przesiadki.

7.3 Główna pętla Tabu Search (`tabu.py`)

```
def tabu_search(start_name, start_ids, candidates, start_time, graph,
criterion,
                max_iterations=100, tabu_size=None, aspiration=False,
sample_size=None):

    n = len(candidates)
    if tabu_size == 'auto':
        tabu_size = 2 * n                                        # (1)

    ordered = greedy_initial(start_ids, candidates, start_time, graph)
    cur_names = [start_name] + [name for name, _ in ordered] +
[start_name]
    cur_ids   = [start_ids]  + [ids  for _, ids  in ordered] + [start_ids]
    cur_cost, cur_results = evaluate_tour(cur_ids, start_time, graph,
criterion)

    best_names, best_ids = cur_names[:], cur_ids[:]
    best_cost, best_results = cur_cost, cur_results

    tabu = deque()
    tabu_set = set()                                             # (2)

    for iteration in range(max_iterations):
        all_swaps = [(i, j) for i in range(1, n+1)
                            for j in range(i+1, n+1)]         # (3)

        if sample_size is not None and len(all_swaps) > sample_size:
            all_swaps = random.sample(all_swaps, sample_size)  # (4)

        best_cand_cost, best_cand_swap = None, None
        best_cand_names, best_cand_ids, best_cand_results = None, None,
None

        for i, j in all_swaps:
            move = frozenset({cur_names[i], cur_names[j]})
            is_tabu = move in tabu_set

            nb_names, nb_ids = cur_names[:], cur_ids[:]
            nb_names[i], nb_names[j] = nb_names[j], nb_names[i]
            nb_ids[i],   nb_ids[j]   = nb_ids[j],   nb_ids[i]

            cost, results = evaluate_tour(nb_ids, start_time, graph,
criterion)
            if results is None:
                continue

            if is_tabu and aspiration and cost < best_cost:    # (5)
                is_tabu = False

            if is_tabu:
```

```
                continue

            if best_cand_cost is None or cost < best_cand_cost:
                best_cand_cost = cost
                best_cand_swap = (i, j)
                best_cand_names, best_cand_ids = nb_names, nb_ids
                best_cand_results = results

        if best_cand_swap is None:
            break

        i, j = best_cand_swap
        move = frozenset({cur_names[i], cur_names[j]})
        cur_names, cur_ids = best_cand_names, best_cand_ids
        cur_cost, cur_results = best_cand_cost, best_cand_results

        tabu.append(move)
        tabu_set.add(move)
        if tabu_size is not None and len(tabu) > tabu_size:    # (6)
            tabu_set.discard(tabu.popleft())

        if cur_cost < best_cost:
            best_cost = cur_cost
            best_names, best_ids = cur_names[:], cur_ids[:]
            best_results = cur_results

    return best_names, best_cost, best_results
```

- **(1)** Wariant (b): automatyczny rozmiar tablicy tabu — `2 * n`. Dla 3 przystanków: rozmiar = 6.
- **(2)** Dwie struktury dla listy tabu: `deque` (FIFO do usuwania) i `set` (O(1) lookup).
- **(3)** Sąsiedztwo swap: wszystkie pary pozycji pośrednich [1..n] — pozycja 0 i n+1 to zawsze start/koniec.
- **(4)** Wariant (d): losujemy `sample_size` kandydatów zamiast sprawdzać wszystkich.
- **(5)** Wariant (c): aspiracja — ruch tabu dopuszczony jeśli bije globalne optimum.
- **(6)** FIFO: gdy lista przepełniona, usuwamy najstarszy ruch z `deque` i `set` jednocześnie.

---

8. Wyniki

Testy wykonano na danych Kolei Dolnośląskich, piątek, odjazd 08:40.

8.1 Lubawka → Rokitki; Wałbrzych Główny, kryterium czasu (`t`)

```
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny" t "8:40" "pt"

Trasa: Lubawka → Wałbrzych Główny → Rokitki → Lubawka

Odcinek 1: Lubawka → Wałbrzych Główny
  Lubawka → Sędzisław        [D66]  09:31:00 → 09:51:00
  Sędzisław → Wałbrzych Główny  [D60]  09:56:00 → 10:17:00
```

```
Odcinek 2: Wałbrzych Główny → Rokitki
  Wałbrzych Główny → Wałbrzych Fabryczny  [D60]  10:18:00 → 10:22:00
  Wałbrzych Fabryczny → Jaworzyna Śląska  [D60]  10:24:00 → 10:53:00
  Jaworzyna Śląska → Legnica  [D91]  10:59:00 → 11:49:00
  Legnica → Chojnów           [D1]   12:14:00 → 12:26:00
  Chojnów → Rokitki           [D14]  13:20:00 → 13:30:00

Odcinek 3: Rokitki → Lubawka
  Rokitki → Legnica           [D13]  14:24:00 → 14:52:00
  Legnica → Jaworzyna Śląska  [D91]  15:01:00 → 15:49:00
  Jaworzyna Śląska → Wałbrzych Szczawienko  [D60]  15:54:00 → 16:10:00
  Wałbrzych Szczawienko → Boguszów-Gorce Wschód  [D60]  16:10:00 →
16:35:00
  Boguszów-Gorce Wschód → Sędzisław  [D60]  16:37:00 → 16:53:00
  Sędzisław → Lubawka         [D66]  17:15:00 → 17:35:00

Odjazd:          09:31:00
Przyjazd:        17:35:00
Czas całkowity:  00:08:04
Przesiadki:      10
```

Greedy wybrał Wałbrzych jako pierwszy przystanek (bliżej Lubawki), potem Rokitki. Tabu Search potwierdził że to optymalna kolejność — zamiana dałaby dłuższą trasę.

8.2 Lubawka → Rokitki; Wałbrzych Główny, kryterium przesiadek (p)

```
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny" p "8:40" "pt"

Trasa: Lubawka → Wałbrzych Główny → Rokitki → Lubawka

Odcinek 1: Lubawka → Wałbrzych Główny
  Lubawka → Sędzisław        [D66]  09:31:00 → 09:51:00
  Sędzisław → Wałbrzych Główny  [D60]  09:56:00 → 10:17:00

Odcinek 2: Wałbrzych Główny → Rokitki
  Wałbrzych Główny → Wrocław Główny  [D60]  10:18:00 → 11:38:00
  Wrocław Główny → Rokitki   [D14]  13:35:00 → 14:44:00

Odcinek 3: Rokitki → Lubawka
  Rokitki → Wrocław Główny   [D14]  17:52:00 → 18:56:00
  Wrocław Główny → Sędzisław [D5/D62]  19:40:00 → 21:33:00
  Sędzisław → Lubawka        [D66]  06:52:00 (+1d) → 07:12:00 (+1d)

Odjazd:          09:31:00
Przyjazd:        07:12:00 (+1d)
Czas całkowity:  00:21:41
Przesiadki:      4
```

Kryterium przesiadek wybiera inne połączenia na każdym odcinku — przez Wrocław zamiast przez Legnicę. Całkowity czas podróży wydłuża się do ponad 21 godzin (przyjazd następnego dnia), ale liczba przesiadek spada z 10 do 4.

8.3 Lubawka → Rokitki; Wałbrzych Główny; Legnica, kryterium czasu (`t`)

```
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny;Legnica" t "8:40"
"pt"

Trasa: Lubawka → Wałbrzych Główny → Legnica → Rokitki → Lubawka

Odcinek 1: Lubawka → Wałbrzych Główny
  Lubawka → Sędzisław        [D66]  09:31:00 → 09:51:00
  Sędzisław → Wałbrzych Główny  [D60]  09:56:00 → 10:17:00

Odcinek 2: Wałbrzych Główny → Legnica
  Wałbrzych Główny → Wałbrzych Fabryczny  [D60]  10:18:00 → 10:22:00
  Wałbrzych Fabryczny → Jaworzyna Śląska  [D60]  10:24:00 → 10:53:00
  Jaworzyna Śląska → Legnica  [D91]  10:59:00 → 11:49:00

Odcinek 3: Legnica → Rokitki
  Legnica → Chojnów  [D1]  12:14:00 → 12:26:00
  Chojnów → Rokitki  [D14]  13:20:00 → 13:30:00

Odcinek 4: Rokitki → Lubawka
  Rokitki → Legnica           [D13]  14:24:00 → 14:52:00
  Legnica → Jaworzyna Śląska  [D91]  15:01:00 → 15:49:00
  Jaworzyna Śląska → Wałbrzych Szczawienko  [D60]  15:54:00 → 16:10:00
  Wałbrzych Szczawienko → Boguszów-Gorce Wschód  [D60]  16:10:00 →
16:35:00
  Boguszów-Gorce Wschód → Sędzisław  [D60]  16:37:00 → 16:53:00
  Sędzisław → Lubawka         [D66]  17:15:00 → 17:35:00

Odjazd:          09:31:00
Przyjazd:        17:35:00
Czas całkowity:  00:08:04
Przesiadki:      9
Czas obliczenia: 0.058s
```

Wejściowa kolejność listy to `Rokitki;Wałbrzych Główny;Legnica`. Greedy zamienił ją na `Wałbrzych → Legnica → Rokitki` — geograficznie logiczna trasa wzdłuż linii D60/D91. Tabu Search potwierdził optymalność tej kolejności.

8.4 Porównanie wariantów (Lubawka → Rokitki; Wałbrzych Główny; Legnica, kryterium `t`)

```
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny;Legnica" t "8:40"
"pt"              → 17:35, 0.058s
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny;Legnica" t "8:40"
```

```
"pt" --tabu-size auto  → 17:35, 0.058s
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny;Legnica" t "8:40"
"pt" --aspiration      → 17:35, 0.061s
$ python main2.py "Lubawka" "Rokitki;Wałbrzych Główny;Legnica" t "8:40"
"pt" --sample 3        → 17:35, 0.061s
```

Wariant	Kolejność	Przyjazd	Czas obliczenia
(a) bazowy	Wałbrzych → Legnica → Rokitki	17:35	0.058s
(b) `--tabu-size auto` (|T|=6)	Wałbrzych → Legnica → Rokitki	17:35	0.058s
(c) `--aspiration`	Wałbrzych → Legnica → Rokitki	17:35	0.061s
(d) `--sample 3`	Wałbrzych → Legnica → Rokitki	17:35	0.061s

Wszystkie warianty dają identyczny wynik — greedy initial solution trafia w optimum globalne dla tej trasy. Warianty (b), (c), (d) są zaprojektowane z myślą o większych instancjach, gdzie przestrzeń rozwiązań ma więcej lokalnych optimów.

8.5 Porównanie wariantów dla 7 przystanków (Wrocław Główny, kryterium `t`)

Trasa: Wrocław Główny → Legnica, Jelenia Góra, Karpacz, Wałbrzych Główny, Świdnica Miasto, Lubin, Głogów, odjazd 08:00, poniedziałek.

Wariant	Kolejność znaleziona	Przyjazd	Czas całkowity	Przesiadki	Czas obliczenia
(a) bazowy	Legnica → Głogów → Lubin → Wałbrzych → Jelenia Góra → Karpacz → Świdnica	20:09	11h 48min	8	3.1s
(b) `--tabu-size auto`	Karpacz → Jelenia Góra → Wałbrzych → Świdnica → Legnica → Lubin → Głogów	**18:57**	**10h 47min**	8	13.9s
(c) `--aspiration`	Karpacz → Jelenia Góra → Wałbrzych → Świdnica → Lubin → Głogów → Legnica	19:02	10h 52min	7	3.2s
(d) `--sample 5`	Wałbrzych → Karpacz → Jelenia Góra → Legnica → Lubin → Głogów → Świdnica	19:37	11h 27min	7	**0.6s**

Wnioski:

- Dla 7 przystanków przestrzeń rozwiązań jest wystarczająco duża, żeby warianty zaczęły się różnić.
- **(b) `--tabu-size auto`** znalazł najkrótszą trasę (10h 47min), ale kosztem 4-krotnie dłuższego czasu obliczenia — dłuższa lista tabu pozwala unikać lokalnych optimów przez więcej iteracji.

- **(c)** `--aspiration` dał zbliżony wynik (10h 52min) przy tym samym czasie co wariant bazowy — kryterium aspiracji pozwoliło wykonać ruch tabu który doprowadził do lepszego globalnego optimum.
- **(d)** `--sample 5` jest najszybszy (0.6s), ale daje gorszy wynik — losowe próbkowanie 5 z 21 możliwych swapów pomija dobre sąsiedztwa i utyka w lokalnym optimum.

8.6 Porównanie wariantów dla 10 przystanków — zachodnia sieć KD (kryterium `t`)

Trasa: Wrocław Główny → Zgorzelec, Bolesławiec, Żagań, Żary, Węgliniec, Szklarska Poręba Górna, Kamienna Góra, Wałbrzych Szczawienko, Strzegom, Legnica, odjazd 08:00, poniedziałek.

Wariant	Przyjazd	Czas całkowity	Przesiadki	Czas obliczenia
(a) bazowy	06:42 (+1d)	22h 32min	9	15.3s
(b) `--tabu-size auto`	06:42 (+1d)	22h 32min	9	15.4s
(c) `--aspiration`	06:42 (+1d)	22h 32min	9	15.5s
(d) `--sample 5`	06:42 (+1d)	22h 32min	9	15.4s

Wszystkie warianty znalazły identyczną trasę: `Wrocław Główny → Wałbrzych Szczawienko → Kamienna Góra → Szklarska Poręba Górna → Strzegom → Żary → Żagań → Bolesławiec → Zgorzelec → Węgliniec → Legnica → Wrocław Główny`

Jest to przypadek gdzie greedy initial solution trafia od razu w optimum globalne — żaden swap w sąsiedztwie nie daje lepszego wyniku. Tabu Search potwierdza optymalność rozwiązania greedy. Układ przystanków geograficznie wzdłuż zachodniej części sieci KD sprawia że naturalna kolejność "po drodze" jest trudna do pobicia.

8.7 Wariant z losowym układem przystanków — `--aspiration` jako wygrywający (kryterium `t`)

Trasa: Wrocław Główny → Karpacz, Żagań, Oleśnica, Lubawka, Głogów, Oława, Jelenia Góra, odjazd 08:00, poniedziałek.

Wariant	Przyjazd	Czas całkowity	Przesiadki	Czas obliczenia
(a) bazowy	08:43 (+1d)	24h 22min	19	4.2s
(b) `--tabu-size auto`	08:43 (+1d)	24h 22min	19	4.8s
(c) `--aspiration`	**06:36 (+1d)**	**22h 26min**	**12**	4.5s
(d) `--sample 5`	08:43 (+1d)	24h 22min	19	3.9s

Warianty (a), (b), (d) utknęły w tym samym lokalnym optimum. Tylko `--aspiration` odblokował kluczowy ruch tabu prowadzący do znacznie lepszego rozwiązania — o 2h krótszego i z 7 mniejszą liczbą przesiadek. Jest to przykład gdzie kryterium aspiracji ma realny wpływ na wynik.

8.8 Deterministyczność algorytmu

Wszystkie warianty (a), (b), (c) są w pełni deterministyczne:

- **`greedy_initial`** — zawsze wybiera przystanek z najwcześniejszym czasem przybycia. Dijkstra jest deterministyczna — te same dane = ten sam wynik.
- **`evaluate_tour`** — sekwencyjne wywołania `search()` z propagacją czasów. Brak losowości.
- **Pętla Tabu Search** — `all_swaps` generowane w stałej kolejności, iteracja deterministyczna, wybór najlepszego kandydata przez `<` na kosztach, lista tabu to `deque` + `set`.

Jedyne źródło losowości to wariant **(d)** **`--sample`** — `random.sample(all_swaps, sample_size)`. W praktyce jednak również daje stałe wyniki, ponieważ ziarno losowości nie jest resetowane między uruchomieniami i przy małej przestrzeni próbkowania Python trafia konsekwentnie w te same kombinacje. Potwierdzono to empirycznie uruchamiając każdy wariant 4-krotnie z identycznymi parametrami — wyniki były identyczne we wszystkich przypadkach.

---

4. Użyte biblioteki

Biblioteka	Zastosowanie
`heapq` (stdlib)	Kolejka priorytetowa w Dijkstrze i A*
`csv` (stdlib)	Parsowanie plików GTFS (.txt)
`datetime` (stdlib)	Obliczanie dnia tygodnia, filtrowanie kalendarza
`math` (stdlib)	Obliczanie odległości euklidesowej dla heurystyki `at`
`collections.defaultdict` (stdlib)	Budowanie grafu sąsiedztwa i grafu odwróconego
`itertools` (stdlib)	Generator liczników jako tiebreaker w kolejce
`collections.deque` (stdlib)	Kolejka FIFO dla listy tabu w Tabu Search
`random` (stdlib)	Próbkowanie sąsiedztwa w wariancie (d) Tabu Search
`folium`	Generowanie interaktywnej mapy HTML z wynikiem trasy

Implementacja nie wymaga żadnych zewnętrznych zależności poza `folium` (wizualizacja).

---

5. Napotkane problemy

Czasy powyżej 24:00 — format GTFS dopuszcza godziny takie jak `25:10:00` dla kursów realizowanych po północy. Standardowa biblioteka `datetime` nie obsługuje takich wartości. Rozwiązanie: reprezentacja czasu jako liczba sekund od północy (`int`), bez użycia obiektów `time`.

Kursy przekraczające północ — kurs odjeżdżający o 23:50 i przyjeżdżający o 00:30 następnego dnia wymaga załadowania rozkładu dla dwóch dni. Rozwiązanie: wczytanie kursów następnego dnia z `time_offset=86400` sekund i dołączenie ich do grafu.

Normalizacja peronów — plik `stops.txt` zawiera zarówno stacje (`location_type=1`) jak i perony (`location_type=0`). Bez normalizacji przesiadka między peronami tej samej stacji byłaby niemożliwa. Rozwiązanie: mapowanie każdego peronu do `parent_station`.

Przystanki tylko do wysiadania — niektóre przystanki mają `pickup_type=1` (brak możliwości wsiadania). Bez filtrowania tworzyłyby krawędzie wychodzące z przystanków, z których pasażer nie może odjechać. Rozwiązanie: pomijanie takich krawędzi przy budowaniu grafu.

Duplikaty w liście L (Zadanie 2) — użytkownik mógł podać ten sam przystanek wielokrotnie lub uwzględnić przystanek startowy w liście L. Powodowało to odcinek `X → X` z kosztem 0 i pustą trasą. Rozwiązanie: deduplikacja listy przed uruchomieniem algorytmu z zachowaniem kolejności, usunięcie przystanku startowego z listy.

Time-dependent TSP (Zadanie 2) — w klasycznym TSP macierz kosztów jest stała. Tu koszt odcinka i→j zależy od czasu przyjazdu do i, który zależy od całej dotychczasowej trasy. Uniemożliwia to prekomputację macierzy. Rozwiązanie: sekwencyjne wywołania `search()` przy każdej ocenie trasy, z propagacją czasów przyjazdu.

---

6. Wnioski i ograniczenia

6.1 Skalowalność na większe sieci

Koleje Dolnośląskie to stosunkowo mała sieć (~200 stacji, kilkadziesiąt linii). Przy skalowaniu do sieci ogólnopolskiej (PKP: ~2500 stacji) lub europejskiej pojawiłyby się następujące problemy:

Rozmiar grafu w pamięci — aktualnie cały graf ładowany jest do pamięci RAM jako `dict[StopId, list[Connection]]`. Dla sieci ogólnopolskiej liczba krawędzi rośnie liniowo z liczbą kursów i przystanków — przy kilku milionach połączeń dziennie może to przekroczyć dostępną pamięć. Rozwiązanie: leniwe ładowanie danych z bazy (np. SQLite), buforowanie tylko aktywnych tras.

Prekomputacja heurystyk — `ats` (odwrócona Dijkstra) i `aps` (BFS po kursach) wykonują preprocessing na całym grafie przed każdym zapytaniem. Dla dużych sieci czas prekomputacji może przewyższyć czas samego wyszukiwania. Rozwiązanie: cache prekomputowanych heurystyk per stacja docelowa.

Wyszukiwanie przystanków po nazwie — `load_stops_by_name()` zwraca wszystkie `stop_id` dla danej nazwy. W dużych sieciach z wieloma przystankami o podobnych nazwach liczba `source_ids` i `target_ids` może rosnąć, co spowalnia inicjalizację kolejki.

6.2 Często zmieniające się dane

Obecna implementacja wczytuje dane GTFS z plików statycznych przy każdym uruchomieniu. W rzeczywistych systemach rozkłady zmieniają się dynamicznie (opóźnienia, odwołania kursów):

Brak obsługi opóźnień — GTFS-RT (Real-Time) to rozszerzenie standardu GTFS o aktualizacje w czasie rzeczywistym. Obecny kod nie obsługuje tego formatu — operuje wyłącznie na danych statycznych. Kurs opóźniony o 20 minut zostanie zaplanowany zgodnie z rozkładem, co może uniemożliwić przesiadkę.

Invalidacja heurystyk — heurystyki `ats` i `aps` są prekomputowane na podstawie aktualnego grafu. Przy zmianie rozkładu (np. odwołanie kursu) heurystyka może przestać być dopuszczalna jeśli kurs który stanowił "dolną granicę" został usunięty. Wymagałoby to ponownego przeliczenia.

Wczytywanie przy starcie — aktualnie `load_connections()` parsuje pliki CSV przy każdym zapytaniu. Przy częstych zmianach lepszym podejściem byłoby trzymanie grafu w pamięci i aktualizowanie tylko zmienionych kursów.

6.3 Ograniczenia modelu

Brak czasu przesiadki — model zakłada że przesiadka zajmuje 0 sekund: pasażer może wysiąść z kursu o `arrival_time = T` i natychmiast wsiąść do kursu z `departure_time = T`. W rzeczywistości przejście między peronami zajmuje czas (szczególnie na dużych stacjach). Rozwiązanie: dodanie minimalnego czasu przesiadki jako parametru, modyfikacja warunku `conn.departure_time >= cost` na `conn.departure_time >= cost + min_transfer_time`.

Brak ceny biletu — optymalizacja uwzględnia tylko czas i liczbę przesiadek. Dodanie kosztu biletu jako kryterium wymagałoby nowej `SearchConfig` z kosztem jako krotką `(cena, czas)` lub `(cena, przesiadki)`.

Heurystyka `aps` dla 2+ przesiadek — testy pokazują że `aps` jest bardziej efektywna niż `ap` dla tras z 1 przesiadką (Wrocław→Karpacz: 16× redukcja węzłów), ale dla tras z 2 przesiadkami odwraca się (Lubawka→Rokitki: 523 vs 243).

Przyczyna: wszystkie kursy odjeżdżające z Lubawki mają `min_transfers ∈ {2, 3}`. Po wejściu na pierwszy kurs:

- `ap`: `h=1`, `f=(0+1)=1`
- `aps`: `h=2`, `f=(0+2)=2`

`ap` utrzymuje `f=1` przez pierwsze dwa kroki — po pierwszej przesiadce kurs bezpośredni do celu daje `f=(1+0)=1`, więc algorytm "przepływa" przez obie przesiadki przy niskim `f`, nie rywalizując z dużą warstwą stanów. `aps` osiąga `f=2` od razu po pierwszym kroku — ale to samo `f=2` mają też wszystkie stany osiągalne po jednej przesiadce na kursach z `min_transfers=1` (648 kursów × wiele przystanków). A* musi przetworzyć całą tę warstwę zanim wyciągnie cel.

Próba naprawy przez **tie-breaking** (preferowanie stanów z wyższym `g` przy tym samym `f`) nie pomogła — zmienia kolejność wewnątrz warstwy `f=2`, ale nie zmniejsza jej rozmiaru. `aps` jest efektywna gdy sieć ma dużo kursów z `min_transfers=0` (heurystyka skutecznie odcina przestrzeń). Gdy wszystkie kursy ze startu mają `min_transfers≥2`, tighter heurystyka paradoksalnie tworzy grubszą warstwę optymalnego `f` i eksploruje więcej węzłów.